

MÉMOIRE DE PROJET · JUILLET 2026

SecureBackup-Chain

Architecture d'un système de sauvegarde décentralisé, chiffré et auditable combinant Hyperledger Fabric (consensus Raft), IPFS distribué et pipeline AES-256 streaming

Hyperledger Fabric 2.5.4

Raft Consensus

IPFS Cluster

AES-256-CBC

SHA-256

JWT · RBAC

SSH · SFTP

Node.js 18 · React 18

Auteur : Yassofd · ourolaongyassine@gmail.com

Table des matières

1. Introduction et contexte

1. Problématique
2. Objectifs de sécurité
3. Stack technologique

2. Architecture de sécurité multicouche

1. Vue d'ensemble des couches
2. Comment les couches se combinent

3. Hyperledger Fabric et consensus Raft

1. Pourquoi une blockchain permissionnée
2. Raft : fonctionnement et garanties
3. Résistance aux pannes et aux attaques
4. MSP, certificats X.509 et politique de chaincode

4. IPFS : stockage distribué et adressage par contenu

1. Content Identifier (CID)
2. IPFS Cluster et Raft
3. Combinaison IPFS + Fabric

5. Pipeline cryptographique AES-256-CBC

1. Chiffrement streaming single-pass
2. Vecteur d'initialisation et IND-CPA
3. Intégrité SHA-256

6. Authentification et contrôle d'accès (JWT · RBAC)

7. Diagramme de cas d'utilisation

8. Diagrammes de séquence

1. Upload local streaming
2. Sauvegarde distante SFTP
3. Planification automatique

9. Code essentiel annoté

1. crypto.js — chiffrement
2. backup-contract.js — chaincode Fabric
3. backups.js — route API

4. sftp.js — transport SFTP

5. scheduler.js — planificateur

10. Conclusion et perspectives

Introduction et contexte

1.1 Problématique

Les systèmes de sauvegarde centralisés souffrent de trois vulnérabilités structurelles :

VULNÉRABILITÉ	IMPACT	RÉPONSE SECUREBACKUP- CHAIN
Point unique de défaillance (SPOF)	Panne du serveur central = perte totale d'accès	IPFS Cluster multi-nœuds + Raft orderers
Absence de preuve d'intégrité	Fichier corrompu ou altéré indétectable	Hash SHA-256 inscrit sur ledger Fabric immuable
Données en clair	Vol du support = vol des données	AES-256-CBC streaming avant tout stockage
Audit falsifiable	Logs supprimables par un administrateur malveillant	Audit horodaté sur blockchain permissionnée

1.2 Objectifs de sécurité

Le système satisfait les cinq propriétés fondamentales de la sécurité de l'information :

PROPRIÉTÉ	MÉCANISME
Confidentialité	AES-256-CBC, clé maître MASTER_KEY jamais exposée sur le réseau

Intégrité	SHA-256 du fichier clair inscrit sur Fabric, vérifiable à tout moment
Disponibilité	IPFS Cluster Raft — 2 nœuds sur 3 suffisent pour pinning et accès
Authenticité	Identité Fabric via certificats X.509 MSP, JWT côté API
Non-répudiation	txId Fabric horodaté cryptographiquement — infalsifiable

1.3 Stack technologique

COMPOSANT	TECHNOLOGIE	RÔLE
Blockchain	Hyperledger Fabric 2.5.4	Ledger immuable, cluster Raft 3 orderers, MSP identité
Stockage distribué	IPFS Kubo + IPFS Cluster	Fichiers chiffrés, adressage CID, réplication automatique
Chiffrement	AES-256-CBC + SHA-256 (Node.js crypto)	Pipeline streaming single-pass, zéro allocation mémoire
API backend	Node.js 18 + Express 4	REST + JWT, busboy multipart streaming, Prisma ORM
Frontend	React 18 + Vite + TailwindCSS	Interface multi-rôles (Admin / Responsable / Auditeur)
Base de données	PostgreSQL 15	Utilisateurs, planifications, ACL, runs
Transport distant	node-ssh + ssh2-sftp-client	SSH (avec shell) et SFTP (sans shell, SFTP-only compatible)
Planificateur	node-cron	Sauvegardes automatiques avec expression cron

Architecture de sécurité multicouche

2.1 Vue d'ensemble des couches

SecureBackup-Chain suit un modèle de **défense en profondeur** (Defense in Depth) : chaque couche constitue une ligne de défense indépendante. La compromission d'une couche ne suffit pas à compromettre le système.

Couche 1 — Authentification et contrôle d'accès (JWT + RBAC)

Tout accès à l'API nécessite un token JWT signé (HS256). Les rôles **admin**, **responsable** et **auditeur** limitent les actions disponibles. Un auditeur ne peut pas créer de sauvegarde ; un responsable ne peut pas gérer les utilisateurs.

Couche 2 — Chiffrement des données au repos (AES-256-CBC)

Chaque fichier est chiffré *avant* de quitter Node.js, avec un IV aléatoire unique par fichier. Même un accès direct aux nœuds IPFS ne révèle que des octets chiffrés — sans MASTER_KEY, les données sont inaccessibles.

Couche 3 — Intégrité et audit immuable (Hyperledger Fabric)

Le hash SHA-256 du fichier clair, le CID IPFS, l'identité du créateur et un horodatage cryptographique sont inscrits sur le ledger. Modifier l'historique exigerait de recalculer la chaîne de hashes de tous les blocs suivants *et* d'obtenir le consensus de 2 des 3 organisations MSP.

Couche 4 — Disponibilité distribuée (IPFS Cluster + Raft)

Les fichiers chiffrés sont distribués sur plusieurs nœuds IPFS. IPFS Cluster utilise son propre consensus Raft pour coordonner les pins. La perte d'un nœud IPFS n'entraîne aucune perte de données.

Couche 5 — Sécurité du transport (TLS + SSH/SFTP)

Toutes les communications Fabric utilisent TLS mutuel (mTLS). Les connexions aux serveurs distants utilisent SSH2 avec clé privée chiffrée au repos (AES-256-GCM, dérivée de MASTER_KEY). Le sous-protocole SFTP n'exécute aucune commande shell — surface d'attaque minimale.

2.2 Comment les couches se combinent pour l'inviolabilité

Scénario attaque 1 — Vol d'un nœud IPFS : L'attaquant obtient les fichiers chiffrés. Sans MASTER_KEY (jamais stockée sur IPFS), le contenu est illisible. Couche 2 tient.

Scénario attaque 2 — Compromission de la base PostgreSQL : L'attaquant voit les métadonnées applicatives et les CIDs. Les données réelles sont sur IPFS (chiffrées). Le ledger Fabric reste intact. Couches 2 et 3 tiennent.

Scénario attaque 3 — Faux audit : Un admin malveillant tente de supprimer une trace d'audit. Les entrées d'audit sont dans le world state Fabric — immuables après commit. Modifier le ledger exige le consensus de 2/3 orderers Raft ET 2/3 organisations MSP. Couche 3 tient.

Scénario attaque 4 — Token JWT volé : L'attaquant peut appeler l'API avec les droits de la victime. Il ne peut uploader que de nouvelles sauvegardes chiffrées — il ne peut pas lire les données existantes (stockées chiffrées sur IPFS). Il ne peut pas falsifier l'historique Fabric. L'action est tracée avec son identité.

Hyperledger Fabric et consensus Raft

3.1 Pourquoi une blockchain permissionnée

Contrairement aux blockchains publiques (Bitcoin, Ethereum), **Hyperledger Fabric** est une blockchain permissionnée : chaque participant doit posséder un certificat X.509 délivré par une autorité de certification de confiance (Fabric CA). Cette architecture apporte trois avantages critiques pour notre système :

PROPRIÉTÉ	BLOCKCHAIN PUBLIQUE	HYPERLEDGER FABRIC
Identité	Pseudonyme (clé publique)	Certificat X.509 — identité vérifiable
Performance	~10 tx/s (Bitcoin), ~15 tx/s (ETH)	~1000+ tx/s selon configuration
Confidentialité	Données publiques par défaut	Canaux privés, collections privées
Consensus	PoW/PoS (énergivore)	Raft CFT — déterministe, rapide
Finalité	Probabiliste (attendre N blocs)	Immédiate après commit

Dans SecureBackup-Chain, le canal **backupchannel** regroupe les organisations Org1, Org2 et Org3. Chaque organisation dispose de son propre MSP (Membership Service Provider) et de son peer. Le chaincode `backup-cc` s'exécute sur les peers de chaque organisation.

3.2 Raft : fonctionnement et garanties

Fabric utilise le protocole **Raft** (Crash Fault Tolerant — CFT) pour le service d'ordering. Raft garantit que tous les nœuds du cluster s'accordent sur la même séquence de transactions, même en présence de pannes.

Les trois rôles Raft

RÔLE	RESPONSABILITÉ	DANS NOTRE CLUSTER
Leader	Reçoit les transactions, les réplique vers les Followers, décide quand "committer"	Orderer Org1 (par défaut), élu dynamiquement
Follower	Reçoit les réplifications du Leader, vote, répond aux clients	Orderers Org2 et Org3
Candidate	État transitoire lors d'une élection	Tout orderer peut devenir Candidate si le Leader est silencieux

Cycle de vie d'une transaction

Lorsque `fabric.submitTransaction('registerBackup', ...)` est appelé depuis l'API :

ÉTAPE	ACTEUR	ACTION
1. Propose	SDK → Peers	La proposition de tx est envoyée à chaque peer endorser. Chaque peer simule le chaincode et retourne son endossement signé.
2. Broadcast	SDK → Orderer Leader	Le SDK collecte les endossements et envoie la tx complète à l'orderer Leader Raft.
3. Réplication	Leader → Followers	Le Leader réplique l'entrée de log vers Org2 et Org3 via <code>AppendEntries</code> .
4. Consensus	Followers → Leader	Org2 et Org3 confirment. Quorum = 2/3 → le Leader commite.
5. Block delivery	Orderers → Peers	Le bloc ordonné est livré à tous les peers du canal.
6. Validation + Commit	Peers	

Chaque peer valide (policy MVCC, signature), écrit dans le ledger. Le world state CouchDB est mis à jour.

Élection du leader

Si le Leader devient injoignable (timeout `HeartbeatTick`), les Followers passent en mode Candidate et déclenchent une élection. Chaque Candidate incrémente son **term** (numéro d'époque) et demande les votes. Le premier à obtenir la majorité (2/3) devient le nouveau Leader. Dans nos tests, l'arrêt de l'orderer Org1 a déclenché une élection et Org2 a été élu Leader au terme 3 en moins de 2 secondes — sans interruption de service.

Tolérance aux pannes : Un cluster de N orderers tolère $(N-1)/2$ pannes simultanées. Avec 3 orderers (Org1, Org2, Org3), le système reste opérationnel avec 1 orderer en panne. Avec 5 orderers, il tolérerait 2 pannes simultanées.

3.3 Résistance aux pannes et aux attaques

Immuabilité du ledger

Chaque bloc Fabric contient le hash du bloc précédent (*previousHash*), formant une chaîne cryptographique. Modifier un bloc historique invaliderait tous les blocs suivants, ce qui serait immédiatement détecté par tout peer qui relit la chaîne. De plus, chaque bloc est signé par le service d'ordering — toute altération post-commit est détectable.

Politique d'endorsement du chaincode

Le chaincode `backup-cc` utilise la politique :

```
OR('Org1MSP.member', 'Org2MSP.member', 'Org3MSP.member')
```

Cela signifie qu'au moins un peer d'une organisation reconnue doit signer chaque transaction. Seuls les membres authentifiés par un MSP valide peuvent enregistrer des sauvegardes. Un attaquant externe sans certificat X.509 valide ne peut pas soumettre de transactions.

Limites de Raft (CFT vs BFT)

Important : Raft est *Crash Fault Tolerant* (CFT), pas *Byzantine Fault Tolerant* (BFT). Il résiste aux pannes (arrêt, crash réseau) mais pas à un orderer malveillant qui enverrait délibérément de fausses données. Dans un contexte bancaire haute sécurité, on envisagerait BFT-SMaRt ou PBFT. Pour SecureBackup-Chain, CFT est approprié car les orderers sont opérés par des organisations de confiance identifiées par MSP.

3.4 MSP, certificats X.509 et politique de chaincode

Le **Membership Service Provider (MSP)** est la composante qui traduit les certificats cryptographiques en identités Fabric. Chaque organisation dispose de son propre MSP avec :

- **Fabric CA** : autorité de certification qui émet les certificats des admins, peers et clients
- **Certificats d'admin** : pour les opérations de gestion (déploiement chaincode, join channel)
- **Certificats de client** : stockés dans le wallet Node.js, signent les transactions
- **TLS CA** : certificats séparés pour les connexions gRPC/TLS entre composants

Dans le chaincode, `ctx.clientIdentity.getID()` retourne l'identité X.509 complète du signataire — infalsifiable car la signature est vérifiée cryptographiquement par chaque peer avant acceptation.

IPFS : stockage distribué et adressage par contenu

4.1 Content Identifier (CID)

IPFS n'adresse pas les fichiers par leur emplacement (où ils sont) mais par leur contenu (*ce qu'ils sont*). Un **CID** (Content Identifier) est un hash cryptographique du contenu du fichier, encodé en base58 :

QmXoypizjW3WknFiJnKLwHCnL72vedxjQkDDP1mXWo6uco — SHA2-256 du contenu, codec dag-pb

Cette propriété est fondamentale pour notre système de sauvegarde :

PROPRIÉTÉ	CONSÉQUENCE
CID = hash(contenu)	Deux fichiers identiques ont le même CID — déduplication automatique
CID immuable	Un fichier modifié a un CID différent — falsification détectable immédiatement
Vérification native	Tout nœud IPFS vérifie que le contenu téléchargé correspond au CID

Quand on inscrit le CID sur Fabric avec le hash SHA-256 du clair, on crée un lien cryptographique à deux niveaux : le CID prouve l'intégrité du *chiffré*, le hash Fabric prouve l'intégrité du *clair original*.

4.2 IPFS Cluster et Raft

IPFS Cluster ajoute une couche d'orchestration au-dessus des nœuds IPFS individuels. Il utilise également **Raft** (implémentation interne `go-libp2p-raft`) pour maintenir une vue cohérente de l'état du cluster entre tous les nœuds :

FONCTION IPFS CLUSTER	MÉCANISME
Pin distribué	Quand l'API demande un pin, le leader Raft réplique la décision à tous les nœuds
Facteur de répllication	Configurable : <code>replication_factor_min/max</code> — un fichier peut être répliqué sur N nœuds
Récupération automatique	Si un nœud rejoint après une panne, il re-synchronise son état depuis le leader

4.3 Combinaison IPFS + Fabric : garantie d'intégrité croisée

La combinaison des deux systèmes crée une garantie d'intégrité que ni l'un ni l'autre ne pourrait offrir seul :

IPFS seul garantit que le fichier téléchargé correspond au CID — mais le CID lui-même pourrait être falsifié dans une base de données classique.

Fabric seul garantit que les métadonnées (hash, taille, propriétaire) sont immuables — mais sans IPFS, les fichiers pourraient être perdus ou modifiés.

IPFS + Fabric ensemble : Le CID stocké sur Fabric est immuable. Le fichier récupéré depuis IPFS est vérifié contre ce CID. Il est donc impossible de substituer un fichier falsifié sans que la vérification échoue — même pour un administrateur système avec accès à toutes les machines.

Pipeline cryptographique AES-256-CBC

5.1 Chiffrement streaming single-pass

La contrainte principale du système est de traiter des fichiers potentiellement très volumineux (plusieurs dizaines de Go) sans saturer la mémoire du serveur. La fonction `createEncryptStream()` résout ce problème avec un **générateur asynchrone** qui produit les chunks chiffrés au fur et à mesure de leur réception :

Flux de données : **busboy** (HTTP multipart) → **createEncryptStream** (AES + SHA256 simultanés) → **ipfs.addFromAsyncIterable** (IPFS) — aucun buffer complet en mémoire, aucune écriture disque intermédiaire.

Le hash SHA-256 est calculé sur le flux *clair* (avant chiffrement) en même temps que le chiffrement, en un seul passage. `getHash()` n'est callable qu'après la consommation complète du flux — appel prématuré retournerait un hash partiel invalide.

5.2 Vecteur d'initialisation et IND-CPA

AES-256-CBC nécessite un **Vecteur d'Initialisation (IV)** de 16 octets. Son rôle est critique :

SCÉNARIO	IV FIXE (MAUVAISE PRATIQUE)	IV ALÉATOIRE PAR FICHIER (NOTRE IMPL.)
Deux fichiers identiques	Chiffrés identiques → révèle l'égalité des clairs	Chiffrés différents → aucune information leakée
Attaque par dictionnaire	Possible si l'attaquant connaît des textes clairs	Impossible — IV inconnu rend chaque chiffré unique
Sécurité sémantique	Non satisfaite (IND-CPA échoue)	Satisfaite — propriété IND-CPA respectée

L'IV est généré par `crypto.randomBytes(16)` (CSPRNG du noyau — `/dev/urandom` sur Linux) et préfixé aux données chiffrées. Le déchiffrement extrait les 16 premiers octets comme IV, puis déchiffre le reste.

5.3 Intégrité SHA-256

Le hash SHA-256 du fichier clair sert de **empreinte d'intégrité** inscrite sur le ledger Fabric. Pour vérifier une sauvegarde :

1. Récupérer le fichier chiffré depuis IPFS (le CID garantit l'intégrité du chiffré)
2. Déchiffrer avec `MASTER_KEY`
3. Calculer SHA-256 du résultat
4. Comparer avec le hash stocké sur Fabric

Si le hash correspond, le fichier est intègre et authentique. Si l'une des deux vérifications échoue (CID ou SHA-256), la corruption ou la falsification est prouvée cryptographiquement.

Authentification et contrôle d'accès (JWT · RBAC)

6.1 JSON Web Token (JWT)

L'API utilise des **JWT signés HS256** (HMAC-SHA256) avec `JWT_SECRET` (variable d'environnement, jamais exposée). Le token contient : `sub` (userId), `role`, `exp` (expiration). Chaque requête présente le token dans l'header `Authorization: Bearer <token>`.

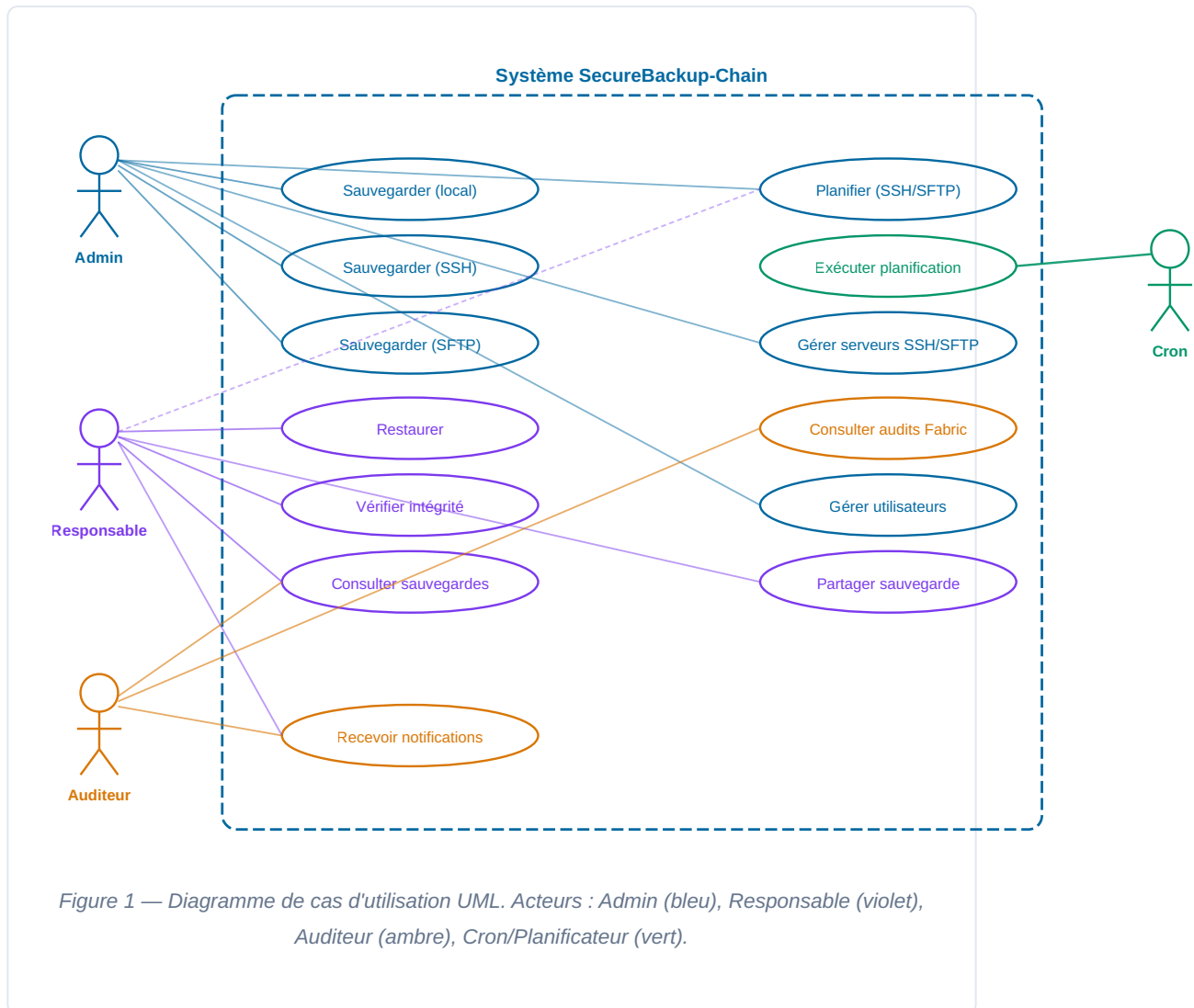
La clé `JWT_SECRET` est distincte de `MASTER_KEY` (chiffrement fichiers). Une compromission de l'une n'implique pas la compromission de l'autre.

6.2 Contrôle d'accès basé sur les rôles (RBAC)

ACTION	ADMIN	RESPONSABLE	AUDITEUR
Créer une sauvegarde	✓	✓	✗
Restaurer une sauvegarde	✓	✓ (siennes)	✗
Consulter les sauvegardes	✓ (toutes)	✓ (siennes)	✓
Gérer les utilisateurs	✓	✗	✗
Configurer SSH/SFTP	✓	✓	✗
Consulter les audits Fabric	✓	✗	✓
Planifier des sauvegardes	✓	✓	✗

Le middleware `requireRole(...roles)` vérifie le champ `role` du JWT décodé et retourne 403 si le rôle n'est pas dans la liste autorisée. La granularité est implémentée au niveau de chaque route Express.

Diagramme de cas d'utilisation



Diagrammes de séquence

8.1 Upload local — streaming AES + IPFS

busboy parse le multipart en streaming et createEncryptStream chiffre et hashé simultanément. Aucun buffer complet en RAM.

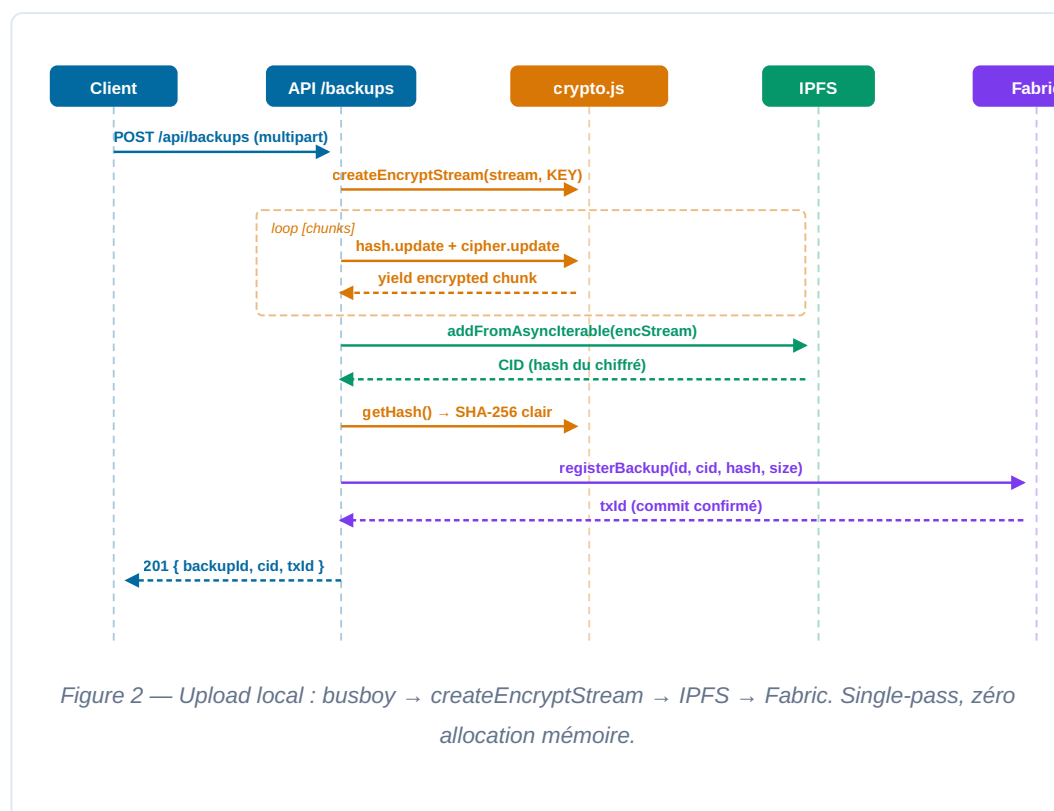
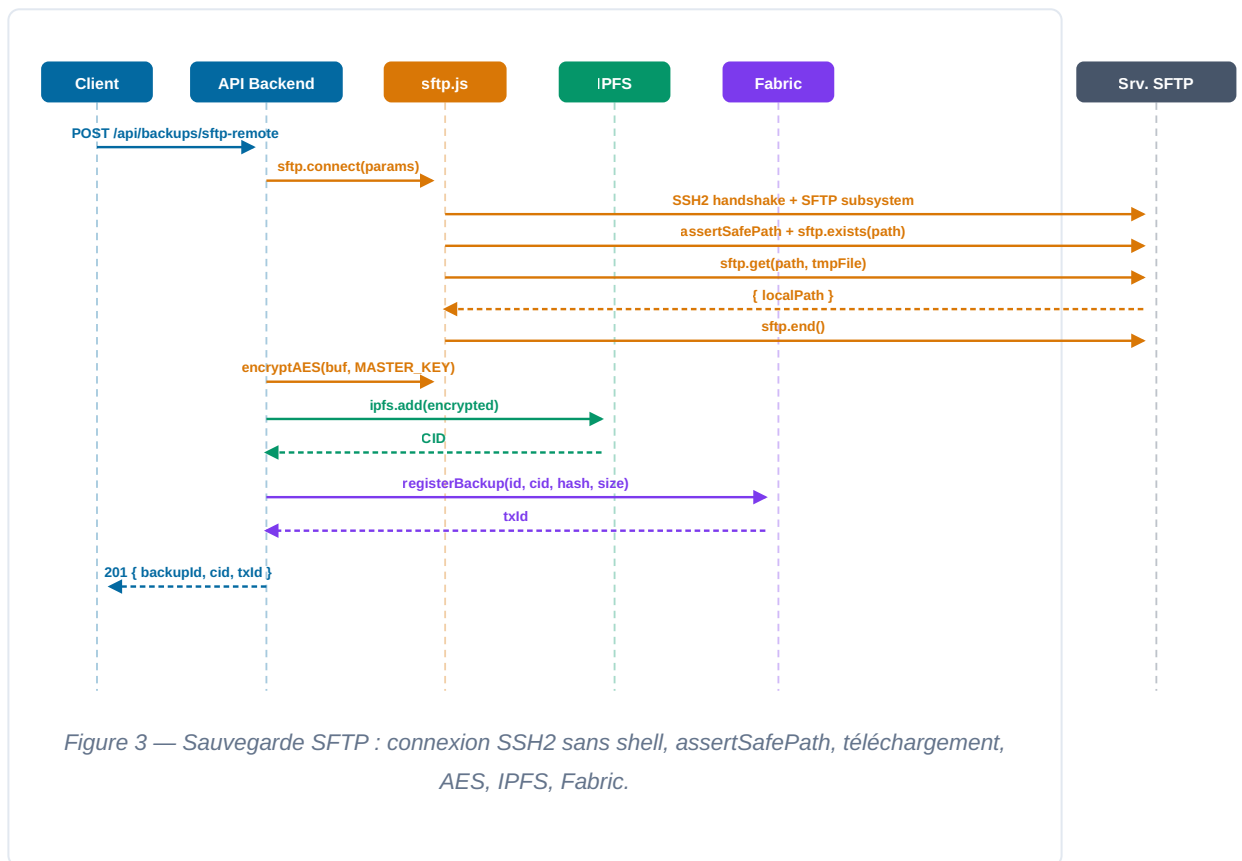
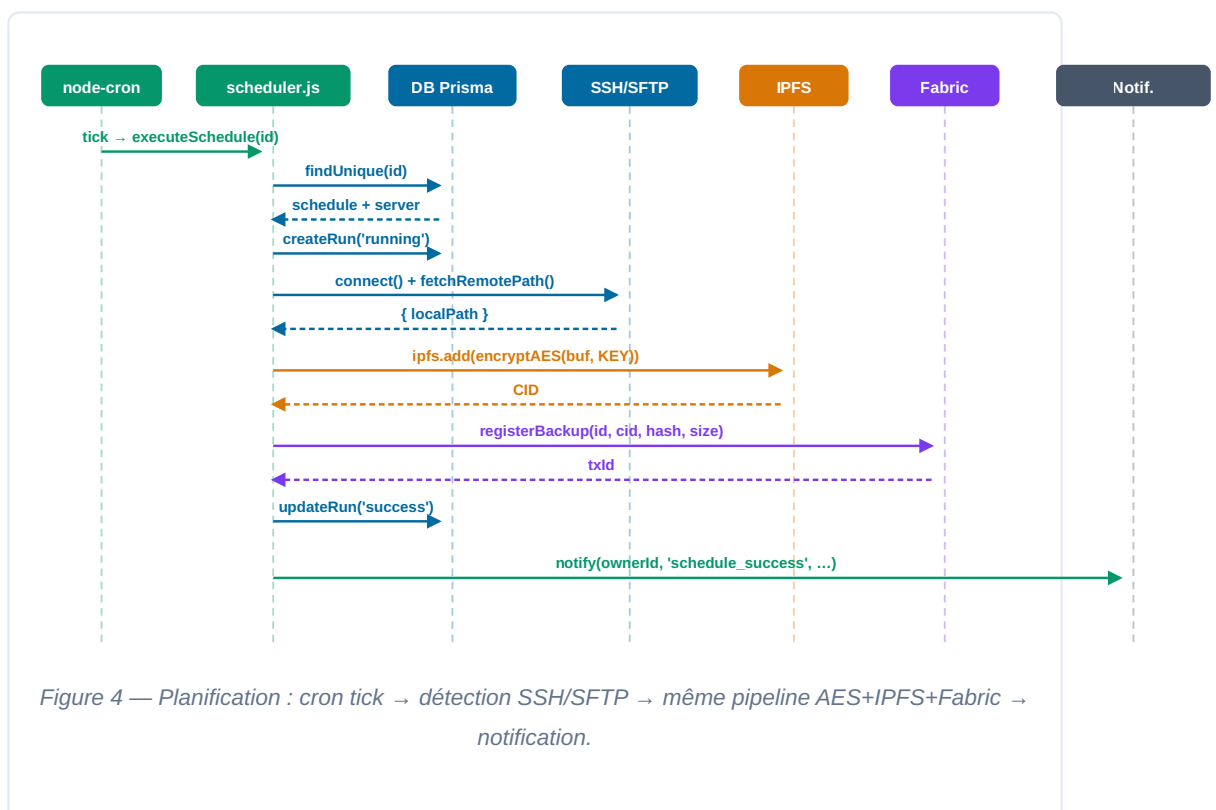


Figure 2 — Upload local : busboy → createEncryptStream → IPFS → Fabric. Single-pass, zéro allocation mémoire.

8.2 Sauvegarde distante SFTP



8.3 Planification automatique (node-cron)



Code essentiel annoté

Cinq fichiers centraux. Les annotations numérotées correspondent aux marqueurs ①②③... dans les commentaires du code.

9.1 backend/src/services/crypto.js — Pipeline de chiffrement

Module central de cryptographie. Implémente AES-256-CBC avec IV aléatoire par fichier et hash SHA-256 simultané du flux clair, en un seul passage mémoire.

```
'use strict';
const crypto = require('crypto');

// ① Hash synchrone pour vérifications ponctuelles
function sha256(buffer) {
  return crypto.createHash('sha256')
    .update(buffer)
    .digest('hex');
}

// ② Chiffrement buffer complet – utilisé par scheduler.js
function encryptAES(buffer, hexKey) {
  const key = Buffer.from(hexKey, 'hex');
  const iv = crypto.randomBytes(16); // IV aléatoire CSPRNG
  const cipher = crypto.createCipheriv('aes-256-cbc', key, iv);
  const enc = Buffer.concat([cipher.update(buffer), cipher.final()]);
  return Buffer.concat([iv, enc]); // IV préfixé aux données
}

// ③ Déchiffrement – extrait l'IV des 16 premiers octets
function decryptAES(buffer, hexKey) {
  const key = Buffer.from(hexKey, 'hex');
  const iv = buffer.subarray(0, 16);
  const ctext = buffer.subarray(16);
  const decipher = crypto.createDecipheriv('aes-256-cbc', key, iv);
  return Buffer.concat([decipher.update(ctext), decipher.final()]);
}

// ④ Streaming single-pass : AES + SHA-256 simultanés
// Utilisé par la route POST /api/backups pour uploads volumineux
function createEncryptStream(inputAsyncIterable, hexKey) {
  const key = Buffer.from(hexKey, 'hex');
  const iv = crypto.randomBytes(16);
  const cipher = crypto.createCipheriv('aes-256-cbc', key, iv);
  const hasher = crypto.createHash('sha256'); // ⑤ hash du CLAIR
  let size = 0;

  async function* gen() {
    yield iv; // IV en tête du flux chiffré
    for await (const chunk of inputAsyncIterable) {
      hasher.update(chunk); // hash avant chiffrement
      size += chunk.length;
      const encrypted = cipher.update(chunk);
      if (encrypted.length > 0) yield encrypted;
    }
    const final = cipher.final(); // padding PKCS7
    if (final.length > 0) yield final;
  }

  return {

```

```

    stream: gen(),
    getHash: () => hasher.digest('hex'), // appeler APRÈS fin du stream
    getSize: () => size,
  };
}

module.exports = { sha256, encryptAES, decryptAES, createEncryptStream };

```

ANNOTATIONS

① sha256(buffer)

Hash synchrone pour vérifications ponctuelles sur des buffers déjà chargés en mémoire (ex. vérification d'intégrité post-restauration).

② encryptAES()

Chiffrement d'un buffer complet. Utilisé par scheduler.js qui lit le fichier distant dans un buffer local avant chiffrement.

③ decryptAES()

L'IV est les 16 premiers octets du buffer chiffré. La clé 256 bits vient de MASTER_KEY (32 octets hex). Décodage PKCS7 automatique par Node.js crypto.

④ createEncryptStream()

Générateur async* pour uploads volumineux. Aucun buffer complet en RAM — fonctionne sur des fichiers de plusieurs Go grâce au backpressure natif des async iterables.

⑤ hasher SHA-256

Calcule le hash du *clair* (avant chiffrement). getHash() n'est valide qu'après consommation complète du générateur — appel prématuré donnerait un hash partiel.

9.2 chaincode/lib/backup-contract.js — Chaincode Fabric

Logique métier s'exécutant sur chaque peer Fabric. Chaque invocation est une transaction ordonnée par Raft, signée X.509, inscrite de façon immuable dans le ledger.

```
const { Contract } = require('fabric-contract-api');

// ① Horodatage déterministe depuis l'en-tête de transaction Fabric
// Identique sur tous les peers – pas d'appel à Date.now()
function txTimestamp(ctx) {
  const t = ctx.stub.getTxTimestamp();
  return new Date(Number(t.seconds) * 1000
    + Math.floor(t.nanos / 1e6))
    .toISOString();
}

class BackupContract extends Contract {

  // ② Audit interne – clé unique par transaction
  async _recordAudit(ctx, action, target, details) {
    const key = `audit_${txTimestamp(ctx)}_${ctx.stub.getTxID()}`;
    await ctx.stub.putState(key,
      Buffer.from(JSON.stringify({
        action, target, details,
        actor: ctx.clientIdentity.getID(), // identité X.509
        timestamp: txTimestamp(ctx),
        txId: ctx.stub.getTxID(),
      })));
  }

  // ③ Point d'entrée principal – inscription d'une sauvegarde
  async registerBackup(ctx, backupId, cid, fileName,
    fileHash, fileSize, mimeType) {

    // ④ Idempotence – refus des doublons
    const existing = await ctx.stub.getState(backupId);
    if (existing?.length > 0)
      throw new Error(`Backup ${backupId} already exists`);

    const entry = {
      backupId, cid, fileName, fileHash,
      fileSize: parseInt(fileSize, 10), // params Fabric = strings
      mimeType,
      ownerId: ctx.clientIdentity.getID(),
      ownerMSP: ctx.clientIdentity.getMSPID(),
      timestamp: txTimestamp(ctx),
      txId: ctx.stub.getTxID(),
      status: 'ACTIVE',
      sharedWith: [],
    };

    // ⑤ Écriture dans le world state CouchDB
    await ctx.stub.putState(backupId,
      Buffer.from(JSON.stringify(entry)));
  }
}
```

```

    await this._recordAudit(ctx, 'BACKUP_REGISTERED',
                             backupId, { cid, fileName });

    // ⑥ Événement Fabric – notifie les clients abonnés
    ctx.stub.setEvent('BackupRegistered',
                      Buffer.from(JSON.stringify({ backupId, cid, fileName })));

    return JSON.stringify(entry);
  }
}

```

ANNOTATIONS

① txTimestamp()

Horodatage depuis l'en-tête de la transaction — fourni par l'orderer Raft, identique sur tous les peers. Utiliser Date.now() serait non-déterministe et causerait des divergences de ledger.

② _recordAudit()

Chaque entrée d'audit est une clé unique (timestamp + txId). L'acteur est l'identité X.509 du signataire — infalsifiable car vérifiée cryptographiquement par chaque peer avant endossement.

③ registerBackup()

Point d'entrée unique pour toutes les sauvegardes (local, SSH, SFTP, planifié). La politique d'endorsement
 OR(Org1MSP.member,
 Org2MSP.member, Org3MSP.member)
 garantit qu'un membre authentifié signe la tx.

④ Idempotence

getState() vérifie l'existence avant écriture. Empêche la double-inscription si le SDK retente après un timeout réseau.

⑤ putState()

Écrit dans le world state CouchDB. La donnée n'est définitive qu'après ordonnancement Raft + validation sur tous les peers du canal.

⑥ setEvent()

Fabric émet l'événement vers les listeners SDK. Permet des réactions en temps réel sans polling du ledger.

9.3 backend/src/routes/backups.js — Route upload zero-copy

Route Express qui orchestre busboy → createEncryptStream → IPFS → Fabric en un pipeline continu. Aucun fichier temporaire créé sur le serveur.


```
router.post('/',
  requireRole('admin', 'responsable'), // ① JWT + RBAC
  async (req, res, next) => {
    try {
      await new Promise((resolve, reject) => {
        const bb = busboy({
          headers: req.headers,
          limits: { files: 1 } // ② Un seul fichier par requête
        });

        bb.on('file', async (_, stream, info) => {
          const { filename, mimeType } = info;
          try {
            // ③ Chiffrement streaming – aucun buffer complet
            const { stream: encStream, getHash, getSize }
              = createEncryptStream(stream, env.MASTER_KEY);

            // ④ Upload IPFS depuis l'async iterable chiffré
            const cid = await ipfs
              .addFromAsyncIterable(encStream, filename);

            const fileHash = getHash(); // valide ici : stream consommé
            const fileSize = getSize();

            // ⑤ Inscription Fabric via SDK
            const backupId = randomUUID();
            const entry = await fabric.submitTransaction(
              'registerBackup', backupId, cid,
              filename, fileHash, String(fileSize), mimeType
            );

            // ⑥ ACL PostgreSQL + notification temps réel
            await db.backupOwnership.create({
              data: { backupId: entry.backupId, userId: req.user.sub }
            });
            notify(req.user.sub, 'backup_success',
              'Sauvegarde créée', `"${filename}" sauvegardé`);

            res.status(201).json({
              backupId: entry.backupId, cid: entry.cid, txId: entry.txId
            });
            resolve();
          } catch (err) {
            stream.destroy();
            reject(err);
          }
        });
        req.pipe(bb);
      });
    }
  });
```

```
    } catch (err) { next(err); }  
  });
```

ANNOTATIONS

① **requireRole()**

Middleware qui décode le JWT et vérifie que le rôle est dans la liste autorisée. Retourne 401 si token absent/expiré, 403 si rôle insuffisant.

② **busboy limits**

Limite à 1 fichier par requête. busboy parse le multipart en streaming — le corps HTTP n'est jamais entièrement bufferisé en mémoire.

③ **createEncryptStream**

SHA-256 et AES calculés simultanément. Le stream busboy est directement "pipé" dans le générateur — aucune écriture disque intermédiaire.

④ **addFromAsyncIterable**

L'API IPFS Kubo accepte un async iterable de chunks. Les données chiffrées sont streamées directement vers le nœud IPFS via HTTP.

⑤ **submitTransaction**

Le SDK Fabric attend la confirmation du commit (2/3 orderers Raft + validation peers).

Retourne seulement après que la tx est définitivement dans le ledger.

⑥ **backupOwnership**

Table PostgreSQL pour les ACL applicatifs.

Fabric stocke qui a créé la sauvegarde (certificat X.509), PostgreSQL stocke qui peut y accéder via l'API.

9.4 backend/src/services/sftp.js — Transport SFTP sécurisé

Utilise uniquement le sous-protocole SFTP (SSH2) — aucune commande shell distante. Compatible avec les serveurs SFTP-only. Protection contre path traversal et fichiers trop volumineux.

```
'use strict';
const SftpClient = require('ssh2-sftp-client');

// ① Liste noire – chemins système interdits
const FORBIDDEN_PATHS = [
  '/etc', '/root', '/var/log/auth.log',
  '/proc', '/sys', '/dev'
];
const MAX_SIZE_BYTES = 10 * 1024 * 1024 * 1024; // 10 Go

// ② Validation du chemin – résout les .. avant contrôle
function assertSafePath(remotePath) {
  const normalized = path.posix.normalize(remotePath);
  for (const forbidden of FORBIDDEN_PATHS) {
    if (normalized === forbidden
      || normalized.startsWith(forbidden + '/'))
      throw Object.assign(
        new Error(`Chemin interdit : ${remotePath}`),
        { status: 403 });
  }
  return normalized;
}

// ③ Connexion SSH2 – supporte clé privée OU mot de passe
async function connect(params) {
  const sftp = new SftpClient();
  await sftp.connect(buildConnectConfig(params));
  return sftp; // fermeture via closeConnection()
}

// ④ Téléchargement sécurisé fichier ou répertoire
async function fetchRemotePath(sftp, remotePath) {
  const safe = assertSafePath(remotePath); // ② bloque path traversal
  const typeResult = await sftp.exists(safe); // 'd' | '-' | false

  if (!typeResult) throw Object.assign(
    new Error(`Chemin introuvable : ${safe}`), { status: 404 });

  const isDirectory = typeResult === 'd';
  if (!isDirectory) {
    const stat = await sftp.stat(safe);
    if (stat.size > MAX_SIZE_BYTES) // ⑤ limite taille
      throw Object.assign(new Error('Fichier > 10 Go'), { status: 413 });
  }

  const tmpFile = path.join(os.tmpdir(),
    `sftp_${Date.now()}_${Math.random().toString(36).slice(2)}.tmp`);

  if (isDirectory) {
    const tmpDir = tmpFile + '_dir';
```

```

    fs.mkdirSync(tmpDir, { recursive: true });
    await sftp.downloadDir(safe, tmpDir); // ⑥ dl récursif sans shell
    execSync(`tar -czf "${tmpFile}" -C "${path.dirname(tmpDir)}" "${path.basename(path.resolve(safe))}"`);
    fs.rmSync(tmpDir, { recursive: true, force: true });
  } else {
    await sftp.get(safe, tmpFile); // ⑦ fichier simple
  }

  return { localPath: tmpFile, isDirectory,
    remoteSize: fs.statSync(tmpFile).size };
}

module.exports = { connect, closeConnection, fetchRemotePath, testConnection };

```

ANNOTATIONS

① FORBIDDEN_PATHS

Liste noire des chemins système sensibles.
Empêche d'exfiltrer /etc/passwd, les clés SSH dans /root/.ssh ou les logs d'authentification.

② assertSafePath()

path.posix.normalize() résout les attaques path traversal (../etc/passwd) avant le contrôle. Retourne 403 si interdit.

③ connect()

ssh2-sftp-client établit SSH2 puis ouvre le sous-système SFTP. Aucune commande shell distante — surface d'attaque minimale, compatible SFTP-only.

④ sftp.exists()

Retourne 'd' (répertoire), '-' (fichier) ou false (absent). Pilote la branche répertoire/fichier sans commande ls distante.

⑤ Limite taille

10 Go maximum. Évite les attaques de saturation disque et mémoire sur le serveur backend.

⑥ downloadDir + tar

Répertoire : téléchargement récursif via SFTP (pas de tar remote), puis tar local. Aucune commande exécutée sur le serveur distant.

9.5 backend/src/services/scheduler.js —

Planificateur unifié SSH/SFTP

Orchestre les sauvegardes automatiques. Détecte dynamiquement le protocole (SSH ou SFTP) via la présence du champ sftpServerId, puis exécute le même pipeline cryptographique que les routes API manuelles.

backend/src/services/scheduler.js
(extraits clés)

node-
cron

SSH +
SFTP

```
const cron = require('node-cron');

// ① Map des tâches cron actives – évite les doublons
const activeTasks = new Map(); // scheduleId → ScheduledTask

async function executeSchedule(scheduleId) {
  // ② Chargement avec les relations sshServer et sftpServer
  const schedule = await db.scheduledBackup.findUnique({
    where: { id: scheduleId },
    include: { sshServer: true, sftpServer: true },
  });
  if (!schedule || schedule.status !== 'active') return;

  // ③ Créer un run "running" avant tout traitement
  const run = await db.scheduledBackupRun.create({
    data: { scheduleId, status: 'running', startedAt: new Date() },
  });

  let conn = null;
  // ④ Détection automatique du protocole
  const useSftp = !!schedule.sftpServerId;
  const svc = useSftp ? sftpService : sshService; // interface commune

  try {
    const server = useSftp ? schedule.sftpServer : schedule.sshServer;
    const creds = JSON.parse(decryptCredentials(server.encryptedCredentials));
    conn = await svc.connect({ ...server, credentials: creds });

    // ⑤ Même interface fetchRemotePath pour SSH et SFTP
    const { localPath } = await svc.fetchRemotePath(conn, schedule.remotePath);
    await svc.closeConnection(conn); conn = null;

    // ⑥ Pipeline identique à l'API manuelle
    const buf = fs.readFileSync(localPath);
    const enc = encryptAES(buf, env.MASTER_KEY);
    const cid = await ipfs.add(enc, fileName);
    const entry = await fabric.submitTransaction(
      'registerBackup', backupId, cid, fileName,
      sha256(buf), String(buf.length), mime);

    await db.scheduledBackupRun.update({
      where: { id: run.id },
      data: { status: 'success', completedAt: new Date(),
        backupId: entry.backupId },
    });
    notify(schedule.ownerId, 'schedule_success', ...);
  } catch (err) {
```

```

    await db.scheduledBackupRun.update({
      where: { id: run.id },
      data: { status: 'error', errorMessage: err.message },
    });
    notifyAdmins('schedule_error', ...);

  } finally {
    // ⑦ Fermeture garantie même si IPFS ou Fabric échoue
    if (conn) { try { await svc.closeConnection(conn); } catch (_) {} }
  }
}

// ⑧ Enregistrement d'une tâche cron
function registerTask(schedule) {
  const existing = activeTasks.get(schedule.id);
  if (existing) existing.stop(); // arrêt avant remplacement
  const task = cron.schedule(schedule.cronExpression,
    () => executeSchedule(schedule.id));
  activeTasks.set(schedule.id, task);
}

```

ANNOTATIONS

① activeTasks Map

Évite les exécutions dupliquées si
registerTask() est appelé plusieurs fois pour le même scheduleId (ex. rechargement à chaud après modification d'une planification).

② findUnique + include

Prisma charge la planification avec son serveur SSH ou SFTP en une seule requête. Un seul des deux (sshServer/sftpServer) est non-null.

③ Run "running"

Créé avant tout traitement. Permet de détecter les runs bloqués (status="running" depuis trop longtemps) lors d'un redémarrage du service.

④ useSftp

Détection par sftpServerId non-null. Une seule ligne de branchement — le reste du code est identique pour les deux protocoles.

⑤ Interface commune

SSH et SFTP exposent connect / fetchRemotePath / closeConnection. Le scheduler n'a pas besoin de connaître le protocole sous-jacent.

⑥ Pipeline identique

encryptAES + ipfs.add + fabric.submitTransaction — même logique que la route API manuelle. Un seul code path pour la sécurité.

⑦ finally

La connexion SSH/SFTP est fermée même si IPFS ou Fabric échouent. Évite les fuites de socket pouvant épuiser le pool de connexions.

⑧ registerTask()

node-cron exécute executeSchedule() selon l'expression cron (ex. "0 2 * * *" = chaque nuit

à 02:00). loadAll() appelle registerTask() pour
chaque planification active au démarrage.

Conclusion et perspectives

Bilan technique

SecureBackup-Chain démontre qu'un système de sauvegarde de niveau production peut être construit en combinant des technologies open source matures selon une architecture de sécurité en profondeur :

COMPOSANT	GARANTIE APPORTÉE	TECHNOLOGIE
Consensus Raft	Ordre total des transactions, résistance aux pannes d'orderers	Hyperledger Fabric 2.5.4 (3 orderers)
Immuabilité ledger	Historique des sauvegardes infalsifiable	World State CouchDB + hash chaîne
Confidentialité	Données illisibles sans MASTER_KEY	AES-256-CBC, IV aléatoire par fichier
Intégrité croisée	Falsification détectable à deux niveaux	CID IPFS + SHA-256 sur ledger
Disponibilité	Panne d'un nœud sans perte de données	IPFS Cluster Raft, facteur de réplication
Non-répudiation	Chaque action liée à une identité X.509	MSP Fabric + JWT côté API
Transport sécurisé	Pas de commande shell distante	SFTP subsystem (ssh2-sftp-client)

Perspectives — Roadmap phases 18–26

PHASE	OBJECTIF	TECHNOLOGIE VISÉE
18	Compression avant chiffrement	zstd niveau 3 (~40% gain taille)
19	Parallélisation uploads	Worker threads Node.js

20	Agent daemon de sauvegarde	Service systemd + IPC
21	Remplacement SSH par stockage objet	MinIO S3 + TLS 1.3
22	Déduplication contenu	Content Defined Chunking (CDC)
23	Gestion des clés HSM	HashiCorp Vault + Transit engine
24	Multi-organisation banque + auditeur	Fabric channels privés
25	Conformité DORA + legal hold	Rétention immuable, journaux légaux
26	Monitoring SLA	Prometheus + Grafana

Phase 23 (HSM Vault) est la prochaine étape de sécurité critique : `MASTER_KEY` serait stockée dans Vault Transit (jamais exposée en mémoire Node.js) et toutes les opérations AES seraient déléguées au HSM, réduisant drastiquement la surface d'attaque sur la clé de chiffrement.